

Sistema de Gestão Comercial em Java com Persistência em MySQL

Versões: JDBC e Hibernate

1. Regras:

- **Quantidade de integrantes: Até 3 pessoas**
- **Valor: 5 pontos**

2. Introdução

O desenvolvimento de sistemas corporativos exige domínio de modelagem orientada a objetos, separação de responsabilidades, persistência de dados e implementação consistente de regras de negócio. Em aplicações reais, o acesso a banco de dados pode ser realizado por diferentes abordagens, entre as quais se destacam **JDBC** e **Hibernate**.

Este roteiro propõe a construção de um **sistema de gestão comercial** com foco em **clientes, produtos, estoque e vendas**, permitindo que o mesmo problema seja resolvido em duas versões técnicas distintas. A proposta favorece a compreensão comparativa entre uma abordagem mais manual e uma abordagem baseada em ORM, sem abandonar a estrutura de domínio e as regras funcionais do sistema.

3. Objetivos

3.1 Objetivo Geral

Desenvolver um sistema em Java para gestão comercial, com cadastro de clientes e produtos, controle de estoque e registro de vendas, implementando a persistência de dados em duas versões: uma com JDBC e outra com Hibernate.

3.2 Objetivos Específicos

- Estruturar o domínio do sistema com classes bem definidas.
- Separar regras de negócio, persistência e interface de execução.
- Aplicar princípios de orientação a objetos em um cenário de negócio real.
- Implementar operações de CRUD para as entidades principais.
- Garantir consistência entre vendas e estoque.

- Comparar, na prática, duas estratégias de persistência em Java.

4. Justificativa

A atividade é justificável por reunir elementos centrais da formação em desenvolvimento de software:

- modelagem de entidades com atributos e comportamentos;
- implementação de regras de negócio;
- manipulação de coleções e relações entre objetos;
- persistência de dados em banco relacional;
- uso de transações e controle de consistência.

Por se tratar de um nível avançado, a atividade exige do estudante não apenas execução operacional, mas também tomada de decisão arquitetural, organização de código e atenção à integridade do sistema.

5. Escopo do Sistema

O sistema deverá contemplar os seguintes módulos:

- **Cliente**
- **Produto**
- **Estoque**
- **Venda**
- **Item de Venda**

6. Regras de Negócio

O sistema deve obedecer às seguintes regras:

- Um cliente **inativo** não pode realizar compras.
- Um produto **inativo** não pode ser incluído em vendas.
- Não é permitido vender quantidade superior ao estoque disponível.
- A confirmação de uma venda deve registrar a operação e reduzir o estoque correspondente.
- O cancelamento de uma venda deve restaurar o estoque dos itens vendidos.

- O valor total da venda deve ser calculado com base nos itens.
- O preço unitário do item deve ser registrado no momento da venda.
- O sistema deve permitir consulta por cliente, produto, período e status da venda.

7. Modelo Conceitual

7.1 Cliente

Representa a pessoa que realiza compras.

Atributos sugeridos:

- id
- nome
- cpf
- email
- telefone
- ativo
- dataCadastro

7.2 Produto

Representa o item comercializado.

Atributos sugeridos:

- id
- nome
- descricao
- precoVenda
- precoCusto
- categoria
- ativo

7.3 Estoque

Representa o saldo disponível de um produto.

Atributos sugeridos:

- id
- produto
- quantidadeDisponivel
- quantidadeMinima
- localizacao
- ultimaAtualizacao

7.4 Venda

Representa a operação comercial realizada.

Atributos sugeridos:

- id
- cliente
- dataVenda
- status
- totalBruto
- desconto
- totalLiquido
- itens

7.5 ItemVenda

Representa cada produto incluído em uma venda.

Atributos sugeridos:

- id
- venda
- produto
- quantidade
- precoUnitario

- subtotal

8. Metodologia de Desenvolvimento

O trabalho deverá ser desenvolvido em etapas:

1. Modelagem do domínio

- criação das classes centrais do sistema;
- implementação de atributos, construtores, getters, setters e regras internas.

2. Camada de persistência

- criação de interfaces DAO;
- implementação posterior em duas versões:
 - JDBC;
 - Hibernate.

3. Camada de serviço

- concentração das regras de negócio;
- validação de dados;
- orquestração das operações entre entidades.

4. Interface de execução

- construção de um menu textual em console;
- integração com os serviços;
- execução dos casos principais.

5. Testes funcionais

- verificação dos fluxos de cadastro, venda, cancelamento e consulta;
- validação da consistência do estoque.

9. Critérios de Avaliação

- organização do código;
- clareza da modelagem;

- fidelidade às regras de negócio;
- funcionamento das operações de CRUD;
- consistência entre venda e estoque;
- uso adequado de orientação a objetos;
- separação entre domínio, serviço e persistência;
- comparação técnica entre JDBC e Hibernate.

10. Estrutura Recomendada do Projeto

```
src/  
├── exception/  
├── model/  
├── dao/  
├── service/  
└── Main.java
```

11. Base de Código para os Alunos

A seguir está uma base formal de implementação.

Os alunos devem completar a camada de persistência nas versões **JDBC** e **Hibernate**, mantendo as mesmas regras de negócio.

11.1 Exceções de Domínio

exception/NegocioException.java

```
package exception;  
  
public class NegocioException extends RuntimeException {  
  
    public NegocioException(String mensagem) {  
        super(mensagem);  
    }  
  
    public NegocioException(String mensagem, Throwable causa) {  
        super(mensagem, causa);  
    }  
}
```

exception/EstoqueInsuficienteException.java

```
package exception;  
  
public class EstoqueInsuficienteException extends NegocioException {
```

```

private final int produtoId;
private final int quantidadeSolicitada;
private final int quantidadeDisponivel;

public EstoqueInsuficienteException(int produtoId,
                                     int quantidadeSolicitada,
                                     int quantidadeDisponivel) {
    super("Estoque insuficiente para o produto ID " + produtoId
          + ". Solicitado: " + quantidadeSolicitada
          + " | Disponível: " + quantidadeDisponivel);
    this.produtoId = produtoId;
    this.quantidadeSolicitada = quantidadeSolicitada;
    this.quantidadeDisponivel = quantidadeDisponivel;
}

public int getProdutoId() {
    return produtoId;
}

public int getQuantidadeSolicitada() {
    return quantidadeSolicitada;
}

public int getQuantidadeDisponivel() {
    return quantidadeDisponivel;
}
}

```

11.2 Enumeração de Status

model/StatusVenda.java

```

package model;

public enum StatusVenda {
    ABERTA,
    CONFIRMADA,
    CANCELADA,
    FATURADA;

    public boolean permiteEdicao() {
        return this == ABERTA;
    }

    public boolean permiteCancelamento() {
        return this == ABERTA || this == CONFIRMADA;
    }
}

```

11.3 Entidades de Domínio

model/Cliente.java

```
package model;

import java.time.LocalDateTime;

public class Cliente {

    private int id;
    private String nome;
    private String cpf;
    private String email;
    private String telefone;
    private boolean ativo;
    private LocalDateTime dataCadastro;

    public Cliente() {
        this.ativo = true;
        this.dataCadastro = LocalDateTime.now();
    }

    public Cliente(String nome, String cpf, String email, String telefone) {
        this();
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
        this.telefone = telefone;
    }

    public boolean podeComprar() {
        return ativo;
    }

    public void inativar() {
        this.ativo = false;
    }

    public void ativar() {
        this.ativo = true;
    }

    public int getId() {
        return id;
    }

    public String getNome() {
        return nome;
    }

    public String getCpf() {
        return cpf;
    }

    public String getEmail() {
```

```

        return email;
    }

    public String getTelefone() {
        return telefone;
    }

    public boolean isAtivo() {
        return ativo;
    }

    public LocalDateTime getDataCadastro() {
        return dataCadastro;
    }

    public void setId(int id) {
        this.id = id;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public void setCpf(String cpf) {
        this.cpf = cpf;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public void setTelefone(String telefone) {
        this.telefone = telefone;
    }

    public void setAtivo(boolean ativo) {
        this.ativo = ativo;
    }

    public void setDataCadastro(LocalDateTime dataCadastro) {
        this.dataCadastro = dataCadastro;
    }

    @Override
    public String toString() {
        return "[" + id + "]" + nome + " | CPF: " + cpf + " | "
            + (ativo ? "ATIVO" : "INATIVO");
    }
}

```

model/Produto.java

```
package model;
```

```
import exception.NegocioException;

public class Produto {

    private int id;
    private String nome;
    private String descricao;
    private double precoVenda;
    private double precoCusto;
    private String categoria;
    private boolean ativo;

    public Produto() {
        this.ativo = true;
    }

    public Produto(String nome, String descricao,
                    double precoVenda, double precoCusto, String categoria) {
        this();
        this.nome = nome;
        this.descricao = descricao;
        this.categoria = categoria;
        setPrecoVenda(precoVenda);
        setPrecoCusto(precoCusto);
    }

    public boolean podeSerVendido() {
        return ativo;
    }

    public double calcularMargemLucro() {
        if (precoCusto == 0) {
            return 0;
        }
        return ((precoVenda - precoCusto) / precoCusto) * 100;
    }

    public void inativar() {
        this.ativo = false;
    }

    public void ativar() {
        this.ativo = true;
    }

    public int getId() {
        return id;
    }

    public String getNome() {
        return nome;
    }
}
```

```
}

public String getDescricao() {
    return descricao;
}

public double getPrecoVenda() {
    return precoVenda;
}

public double getPrecoCusto() {
    return precoCusto;
}

public String getCategoria() {
    return categoria;
}

public boolean isAtivo() {
    return ativo;
}

public void setId(int id) {
    this.id = id;
}

public void setNome(String nome) {
    this.nome = nome;
}

public void setDescricao(String descricao) {
    this.descricao = descricao;
}

public void setPrecoVenda(double precoVenda) {
    if (precoVenda <= 0) {
        throw new NegocioException("Preço de venda deve ser maior que
zero.");
    }
    this.precoVenda = precoVenda;
}

public void setPrecoCusto(double precoCusto) {
    if (precoCusto < 0) {
        throw new NegocioException("Preço de custo não pode ser
negativo.");
    }
    if (precoCusto > precoVenda) {
        throw new NegocioException("Preço de custo não pode ser maior
que o preço de venda.");
    }
    this.precoCusto = precoCusto;
}
```

```

    }

    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public void setAtivo(boolean ativo) {
        this.ativo = ativo;
    }

    @Override
    public String toString() {
        return "[" + id + "]" + " " + nome + " | R$ " + precoVenda + " | "
            + categoria + " | " + (ativo ? "ATIVO" : "INATIVO");
    }
}

```

model/Estoque.java

```

package model;

import exception.EstoqueInsuficienteException;
import exception.NegocioException;

import java.time.LocalDateTime;

public class Estoque {

    private int id;
    private Produto produto;
    private int quantidadeDisponivel;
    private int quantidadeMinima;
    private String localizacao;
    private LocalDateTime ultimaAtualizacao;

    public Estoque() {
    }

    public Estoque(Produto produto, int quantidadeDisponivel,
        int quantidadeMinima, String localizacao) {
        this.produto = produto;
        this.quantidadeMinima = quantidadeMinima;
        this.localizacao = localizacao;
        this.ultimaAtualizacao = LocalDateTime.now();
        setQuantidadeDisponivel(quantidadeDisponivel);
    }

    public void registrarEntrada(int quantidade) {
        if (quantidade <= 0) {
            throw new NegocioException("Quantidade de entrada deve ser maior
que zero.");
        }
        this.quantidadeDisponivel += quantidade;
    }
}

```

```
        this.ultimaAtualizacao = LocalDateTime.now();
    }

    public void registrarSaida(int quantidade) {
        if (quantidade <= 0) {
            throw new NegocioException("Quantidade de saída deve ser maior
que zero.");
        }
        if (quantidade > quantidadeDisponivel) {
            throw new EstoqueInsuficienteException(
                produto.getId(), quantidade, quantidadeDisponivel);
        }
        this.quantidadeDisponivel -= quantidade;
        this.ultimaAtualizacao = LocalDateTime.now();
    }

    public boolean estaBaixoDoMinimo() {
        return quantidadeDisponivel < quantidadeMinima;
    }

    public boolean temDisponibilidade(int quantidade) {
        return quantidadeDisponivel >= quantidade;
    }

    public int getId() {
        return id;
    }

    public Produto getProduto() {
        return produto;
    }

    public int getQuantidadeDisponivel() {
        return quantidadeDisponivel;
    }

    public int getQuantidadeMinima() {
        return quantidadeMinima;
    }

    public String getLocalizacao() {
        return localizacao;
    }

    public LocalDateTime getUltimaAtualizacao() {
        return ultimaAtualizacao;
    }

    public void setId(int id) {
        this.id = id;
    }
}
```

```

public void setProduto(Produto produto) {
    this.produto = produto;
}

public void setQuantidadeDisponivel(int quantidadeDisponivel) {
    if (quantidadeDisponivel < 0) {
        throw new NegocioException("Estoque não pode ser negativo.");
    }
    this.quantidadeDisponivel = quantidadeDisponivel;
}

public void setQuantidadeMinima(int quantidadeMinima) {
    this.quantidadeMinima = quantidadeMinima;
}

public void setLocalizacao(String localizacao) {
    this.localizacao = localizacao;
}

public void setUltimaAtualizacao(LocalDateTime ultimaAtualizacao) {
    this.ultimaAtualizacao = ultimaAtualizacao;
}

@Override
public String toString() {
    return "Estoque [" + produto.getNome() + "] | Disponível: "
        + quantidadeDisponivel + " | Mínimo: " + quantidadeMinima;
}
}

```

model/ItemVenda.java

```

package model;

import exception.NegocioException;

public class ItemVenda {

    private int id;
    private Venda venda;
    private Produto produto;
    private int quantidade;
    private double precoUnitario;
    private double subtotal;

    public ItemVenda() {
    }

    public ItemVenda(Produto produto, int quantidade) {
        if (!produto.podeSerVendido()) {
            throw new NegocioException("Produto inativo não pode ser
vendido.");
        }
    }
}

```

```
        if (quantidade <= 0) {
            throw new NegocioException("Quantidade do item deve ser maior
que zero.");
        }
        this.produto = produto;
        this.quantidade = quantidade;
        this.precoUnitario = produto.getPrecoVenda();
        calcularSubtotal();
    }

    public void calcularSubtotal() {
        this.subtotal = quantidade * precoUnitario;
    }

    public int getId() {
        return id;
    }

    public Venda getVenda() {
        return venda;
    }

    public Produto getProduto() {
        return produto;
    }

    public int getQuantidade() {
        return quantidade;
    }

    public double getPrecoUnitario() {
        return precoUnitario;
    }

    public double getSubtotal() {
        return subtotal;
    }

    public void setId(int id) {
        this.id = id;
    }

    public void setVenda(Venda venda) {
        this.venda = venda;
    }

    public void setProduto(Produto produto) {
        this.produto = produto;
    }

    public void setQuantidade(int quantidade) {
        if (quantidade <= 0) {
```

```

        throw new NegocioException("Quantidade do item deve ser maior
que zero.");
    }
    this.quantidade = quantidade;
    calcularSubtotal();
}

public void setPrecoUnitario(double precoUnitario) {
    this.precoUnitario = precoUnitario;
    calcularSubtotal();
}

public void setSubtotal(double subtotal) {
    this.subtotal = subtotal;
}

@Override
public String toString() {
    return produto.getNome() + " | Qtd: " + quantidade
        + " | Unit: R$ " + precoUnitario
        + " | Subtotal: R$ " + subtotal;
}
}

```

model/Venda.java

```

package model;

import exception.NegocioException;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Venda {

    private int id;
    private Cliente cliente;
    private LocalDateTime dataVenda;
    private StatusVenda status;
    private double totalBruto;
    private double desconto;
    private double totalLiquido;
    private List<ItemVenda> itens;

    public Venda() {
        this.itens = new ArrayList<>();
        this.status = StatusVenda.ABERTA;
        this.dataVenda = LocalDateTime.now();
    }

    public Venda(Cliente cliente) {

```

```
        this();
        if (!cliente.podeComprar()) {
            throw new NegocioException("Cliente inativo não pode realizar
compras.");
        }
        this.cliente = cliente;
    }

    public void adicionarItem(ItemVenda item) {
        if (!status.permiteEdicao()) {
            throw new NegocioException("Venda no status " + status + " não
permite edição.");
        }
        item.setVenda(this);
        itens.add(item);
        recalcularTotais();
    }

    public void removerItem(int indice) {
        if (!status.permiteEdicao()) {
            throw new NegocioException("Venda no status " + status + " não
permite edição.");
        }
        if (indice < 0 || indice >= itens.size()) {
            throw new NegocioException("Índice de item inválido.");
        }
        itens.remove(indice);
        recalcularTotais();
    }

    public void aplicarDesconto(double desconto) {
        if (desconto < 0) {
            throw new NegocioException("Desconto não pode ser negativo.");
        }
        if (desconto > totalBruto) {
            throw new NegocioException("Desconto não pode ser maior que o
total bruto.");
        }
        this.desconto = desconto;
        recalcularTotais();
    }

    public void recalcularTotais() {
        this.totalBruto =
itens.stream().mapToDouble(ItemVenda::getSubtotal).sum();
        this.totalLiquido = Math.max(0, totalBruto - desconto);
    }

    public void confirmar() {
        if (itens.isEmpty()) {
            throw new NegocioException("Não é possível confirmar venda sem
itens.");
        }
    }
}
```

```
        if (!status.permiteEdicao()) {
            throw new NegocioException("Venda não pode ser confirmada no
status " + status);
        }
        this.status = StatusVenda.CONFIRMADA;
    }

    public void cancelar() {
        if (!status.permiteCancelamento()) {
            throw new NegocioException("Venda não pode ser cancelada no
status " + status);
        }
        this.status = StatusVenda.CANCELADA;
    }

    public int getId() {
        return id;
    }

    public Cliente getCliente() {
        return cliente;
    }

    public LocalDateTime getDataVenda() {
        return dataVenda;
    }

    public StatusVenda getStatus() {
        return status;
    }

    public double getTotalBruto() {
        return totalBruto;
    }

    public double getDesconto() {
        return desconto;
    }

    public double getTotalLiquido() {
        return totalLiquido;
    }

    public List<ItemVenda> getItens() {
        return Collections.unmodifiableList(itens);
    }

    public void setId(int id) {
        this.id = id;
    }

    public void setCliente(Cliente cliente) {
```

```

        this.cliente = cliente;
    }

    public void setDataVenda(LocalDateTime dataVenda) {
        this.dataVenda = dataVenda;
    }

    public void setStatus(StatusVenda status) {
        this.status = status;
    }

    public void setTotalBruto(double totalBruto) {
        this.totalBruto = totalBruto;
    }

    public void setDesconto(double desconto) {
        this.desconto = desconto;
    }

    public void setTotalLiquido(double totalLiquido) {
        this.totalLiquido = totalLiquido;
    }

    public void setItens(List<ItemVenda> itens) {
        this.itens = itens;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("Venda #").append(id)
          .append(" | Cliente: ").append(cliente.getNome())
          .append(" | Status: ").append(status)
          .append(" | Data: ").append(dataVenda).append("\n");
        for (ItemVenda item : itens) {
            sb.append("  ").append(item).append("\n");
        }
        sb.append("  Total Bruto: R$ ").append(totalBruto)
          .append(" | Desconto: R$ ").append(desconto)
          .append(" | Total Líquido: R$ ").append(totalLiquido);
        return sb.toString();
    }
}

```

11.4 Contratos de Persistência

As interfaces abaixo definem o contrato comum das duas versões.

A implementação concreta deverá ser feita em duas variantes:

- uma com **JDBC**;

- outra com **Hibernate**.

dao/ClienteDAO.java

```
package dao;

import model.Cliente;
import java.util.List;

public interface ClienteDAO {

    void inserir(Cliente cliente);

    void atualizar(Cliente cliente);

    Cliente buscarPorId(int id);

    List<Cliente> listarTodos();

    List<Cliente> listarPorStatus(boolean ativo);

    boolean existeCpf(String cpf);

    void deletar(int id);
}
```

dao/ProdutoDAO.java

```
package dao;

import model.Produto;
import java.util.List;

public interface ProdutoDAO {

    void inserir(Produto produto);

    void atualizar(Produto produto);

    Produto buscarPorId(int id);

    List<Produto> listarTodos();

    List<Produto> listarPorStatus(boolean ativo);

    List<Produto> listarPorCategoria(String categoria);

    void deletar(int id);
}
```

dao/EstoqueDAO.java

```
package dao;
```

```

import model.Estoque;
import java.util.List;

public interface EstoqueDAO {

    void inserir(Estoque estoque);

    void atualizar(Estoque estoque);

    Estoque buscarPorProduto(int produtoId);

    List<Estoque> listarTodos();

    List<Estoque> listarAbaixoDoMinimo();
}

```

dao/VendaDAO.java

```

package dao;

import model.StatusVenda;
import model.Venda;

import java.time.LocalDateTime;
import java.util.List;

public interface VendaDAO {

    void inserir(Venda venda);

    void atualizarStatus(Venda venda);

    Venda buscarPorId(int id);

    List<Venda> listarPorCliente(int clienteId);

    List<Venda> listarPorPeriodo(LocalDateTime inicio, LocalDateTime fim);

    List<Venda> listarPorStatus(StatusVenda status);
}

```

11.5 Camada de Serviço

service/ClienteService.java

```

package service;

import dao.ClienteDAO;
import exception.NegocioException;
import model.Cliente;

import java.util.List;
import java.util.regex.Pattern;

```

```

public class ClienteService {

    private static final Pattern REGEX_EMAIL =
        Pattern.compile("^[\\w.-]+@[\\w.-]+\\. [a-zA-Z]{2,}$");

    private static final Pattern REGEX_CPF =
        Pattern.compile("^\\d{3}\\.\\.\\d{3}\\.\\.\\d{3}-\\d{2}$");

    private final ClienteDAO dao;

    public ClienteService(ClienteDAO dao) {
        this.dao = dao;
    }

    public void cadastrar(Cliente cliente) {
        validar(cliente);
        if (dao.existeCpf(cliente.getCpf())) {
            throw new NegocioException("Já existe um cliente com o CPF
informado.");
        }
        dao.inserir(cliente);
    }

    public void atualizar(Cliente cliente) {
        validar(cliente);
        dao.atualizar(cliente);
    }

    public void inativar(int id) {
        Cliente cliente = buscarPorId(id);
        cliente.inativar();
        dao.atualizar(cliente);
    }

    public void ativar(int id) {
        Cliente cliente = buscarPorId(id);
        cliente.ativar();
        dao.atualizar(cliente);
    }

    public Cliente buscarPorId(int id) {
        Cliente cliente = dao.buscarPorId(id);
        if (cliente == null) {
            throw new NegocioException("Cliente com ID " + id + " não
encontrado.");
        }
        return cliente;
    }

    public List<Cliente> listarTodos() {
        return dao.listarTodos();
    }
}

```

```

    }

    public List<Cliente> listarAtivos() {
        return dao.listarPorStatus(true);
    }

    private void validar(Cliente cliente) {
        if (cliente.getNome() == null || cliente.getNome().isBlank()) {
            throw new NegocioException("Nome do cliente é obrigatório.");
        }
        if (!REGEX_CPF.matcher(cliente.getCpf()).matches()) {
            throw new NegocioException("CPF inválido.");
        }
        if (!REGEX_EMAIL.matcher(cliente.getEmail()).matches()) {
            throw new NegocioException("E-mail inválido.");
        }
    }
}

```

service/ProdutoService.java

```

package service;

import dao.EstoqueDAO;
import dao.ProdutoDAO;
import exception.NegocioException;
import model.Estoque;
import model.Produto;

import java.util.List;

public class ProdutoService {

    private final ProdutoDAO produtoDAO;
    private final EstoqueDAO estoqueDAO;

    public ProdutoService(ProdutoDAO produtoDAO, EstoqueDAO estoqueDAO) {
        this.produtoDAO = produtoDAO;
        this.estoqueDAO = estoqueDAO;
    }

    public void cadastrar(Produto produto, int quantidadeInicial,
                          int quantidadeMinima, String localizacao) {
        validar(produto);
        produtoDAO.inserir(produto);
        Estoque estoque = new Estoque(produto, quantidadeInicial,
quantidadeMinima, localizacao);
        estoqueDAO.inserir(estoque);
    }

    public void atualizar(Produto produto) {
        validar(produto);
        produtoDAO.atualizar(produto);
    }
}

```

```

    }

    public void inativar(int id) {
        Produto produto = buscarPorId(id);
        produto.inativar();
        produtoDAO.atualizar(produto);
    }

    public void ativar(int id) {
        Produto produto = buscarPorId(id);
        produto.ativar();
        produtoDAO.atualizar(produto);
    }

    public Produto buscarPorId(int id) {
        Produto produto = produtoDAO.buscarPorId(id);
        if (produto == null) {
            throw new NegocioException("Produto com ID " + id + " não encontrado.");
        }
        return produto;
    }

    public List<Produto> listarTodos() {
        return produtoDAO.listarTodos();
    }

    public List<Produto> listarAtivos() {
        return produtoDAO.listarPorStatus(true);
    }

    private void validar(Produto produto) {
        if (produto.getNome() == null || produto.getNome().isBlank()) {
            throw new NegocioException("Nome do produto é obrigatório.");
        }
        if (produto.getPrecoVenda() <= 0) {
            throw new NegocioException("Preço de venda deve ser maior que zero.");
        }
        if (produto.getPrecoCusto() > produto.getPrecoVenda()) {
            throw new NegocioException("Preço de custo não pode ser maior que o preço de venda.");
        }
    }
}

```

service/EstoqueService.java

```

package service;

import dao.EstoqueDAO;
import exception.NegocioException;
import model.Estoque;

```

```

import java.util.List;

public class EstoqueService {

    private final EstoqueDAO dao;

    public EstoqueService(EstoqueDAO dao) {
        this.dao = dao;
    }

    public void registrarEntrada(int produtoId, int quantidade) {
        Estoque estoque = buscarPorProduto(produtoId);
        estoque.registrarEntrada(quantidade);
        dao.atualizar(estoque);
    }

    public void registrarSaida(int produtoId, int quantidade) {
        Estoque estoque = buscarPorProduto(produtoId);
        estoque.registrarSaida(quantidade);
        dao.atualizar(estoque);
    }

    public Estoque buscarPorProduto(int produtoId) {
        Estoque estoque = dao.buscarPorProduto(produtoId);
        if (estoque == null) {
            throw new NegocioException("Estoque não encontrado para o
produto informado.");
        }
        return estoque;
    }

    public List<Estoque> listarAbaixoDoMinimo() {
        return dao.listarAbaixoDoMinimo();
    }

    public List<Estoque> listarTodos() {
        return dao.listarTodos();
    }
}

```

service/VendaService.java

```

package service;

import dao.VendaDAO;
import exception.NegocioException;
import model.*;

import java.time.LocalDateTime;
import java.util.List;

public class VendaService {

```

```
private final VendaDAO vendaDAO;
private final EstoqueService estoqueService;

public VendaService(VendaDAO vendaDAO, EstoqueService estoqueService) {
    this.vendaDAO = vendaDAO;
    this.estoqueService = estoqueService;
}

public Venda abrirVenda(Cliente cliente) {
    if (!cliente.podeComprar()) {
        throw new NegocioException("Cliente inativo não pode realizar
compras.");
    }
    return new Venda(cliente);
}

public void adicionarItem(Venda venda, Produto produto, int quantidade)
{
    if (!produto.podeSerVendido()) {
        throw new NegocioException("Produto inativo não pode ser
vendido.");
    }
    Estoque estoque = estoqueService.buscarPorProduto(produto.getId());
    if (!estoque.temDisponibilidade(quantidade)) {
        throw new NegocioException("Estoque insuficiente para o produto
" + produto.getNome());
    }
    venda.adicionarItem(new ItemVenda(produto, quantidade));
}

public void confirmarVenda(Venda venda) {
    if (venda.getItems().isEmpty()) {
        throw new NegocioException("Venda não possui itens.");
    }

    venda.confirmar();

    // TODO: o aluno deverá integrar esta operação à implementação
    // de persistência escolhida, garantindo consistência transacional
    // entre a gravação da venda, dos itens e da baixa de estoque.
    vendaDAO.inserir(venda);

    for (ItemVenda item : venda.getItems()) {
        estoqueService.registrarSaida(item.getProduto().getId(),
item.getQuantidade());
    }
}

public void cancelarVenda(int vendaId) {
    Venda venda = buscarPorId(vendaId);
    venda.cancelar();
}
```

```

        // TODO: o aluno deverá integrar esta operação à implementação
        // de persistência escolhida, garantindo reversão adequada
        // da venda e do estoque.
        vendaDAO.atualizarStatus(venda);

        for (ItemVenda item : venda.getItems()) {
            estoqueService.registrarEntrada(item.getProduto().getId(),
            item.getQuantidade());
        }
    }

    public Venda buscarPorId(int id) {
        Venda venda = vendaDAO.buscarPorId(id);
        if (venda == null) {
            throw new NegocioException("Venda com ID " + id + " não
            encontrada.");
        }
        return venda;
    }

    public List<Venda> listarPorCliente(int clienteId) {
        return vendaDAO.listarPorCliente(clienteId);
    }

    public List<Venda> listarPorPeriodo(LocalDateTime inicio, LocalDateTime
    fim) {
        return vendaDAO.listarPorPeriodo(inicio, fim);
    }

    public List<Venda> listarPorStatus(StatusVenda status) {
        return vendaDAO.listarPorStatus(status);
    }
}

```

11.6 Classe Principal

Main.java

```

import model.*;

import java.util.Scanner;

/**
 * Ponto de entrada da aplicação.
 * A implementação do menu deve ser concluída pelos alunos.
 * Nesta etapa, o foco é conectar a interface com os serviços.
 */
public class Main {

    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.println("SISTEMA DE GESTÃO COMERCIAL");
    }
}

```

```

        System.out.println("Execução iniciada.");
        // TODO: implementar o menu principal e os fluxos de operação.
    }

    private static String lerTexto(String mensagem) {
        System.out.print(mensagem);
        return scanner.nextLine();
    }

    private static int lerInteiro(String mensagem) {
        System.out.print(mensagem);
        return Integer.parseInt(scanner.nextLine());
    }

    private static double lerDouble(String mensagem) {
        System.out.print(mensagem);
        return Double.parseDouble(scanner.nextLine());
    }
}

```

12. Orientações para a Implementação das Versões

12.1 Versão JDBC

Na versão JDBC, o estudante deverá implementar:

- conexão com o banco de dados;
- comandos SQL para inserção, atualização, exclusão e consulta;
- mapeamento manual entre linhas do banco e objetos Java;
- controle de transações nas operações de venda e cancelamento.

12.2 Versão Hibernate

Na versão Hibernate, o estudante deverá implementar:

- configuração da persistência;
- mapeamento das entidades;
- uso de sessões e transações;
- consultas orientadas a objetos;
- persistência e recuperação de relacionamentos entre entidades.

13. Entregáveis Esperados

O trabalho deverá conter:

- código-fonte completo do sistema;
- implementação da versão JDBC;
- implementação da versão Hibernate;
- relatório descritivo com análise comparativa entre as duas abordagens;
- evidência de execução dos principais fluxos do sistema.

14. Critérios Técnicos de Conclusão

O sistema será considerado satisfatório quando apresentar:

- cadastro e consulta de clientes;
- cadastro e consulta de produtos;
- controle de estoque;
- criação de vendas com itens;
- baixa de estoque ao confirmar venda;
- reversão de estoque ao cancelar venda;
- consultas por cliente, status e período;
- consistência entre as entidades do domínio.